



Helwan university
Faculty of Engineering
Computer and systems Engineering Department

MetroM
Cairo Metro Route Planner

Course supervisors:

Dr. Ahmed Hussien | Eng. Rania Elsayed

Team members:

Mina Magdy Ezzat

Hagar Khaled Helmy

Moslem Sayed Shehata

Mohammed Salem Mohammed

Hossam-eldin Omar Mohammed

Table of Contents

1. Introduction.....	4
1.1 Project Objectives	4
1.2 Target Platform	5
2. Section 01: UX Phase	5
2.1 Stage I: Research.....	5
2.1.1 Survey Analysis	5
2.1.2 Empathy Map	7
2.1.3 User Persona	7
2.2 Stage II: Define Problem	8
2.2.1 Pain Points Categorization.....	8
2.2.2 5Ws and H Analysis	8
2.2.3 Problem Statements.....	8
2.3 Stage III: Ideate Solution	9
2.3.1 Goal Statements	9
2.3.2 Proposed Solutions Comparison	9
2.3.3 Comparison with Existing Solutions	10
2.3.4 Selected Solution and App Features.....	10
2.4 Stage IV: Wireframes and Site Map	11
2.4.1 Wireframes.....	11
2.4.2 Site Map and Navigation	17
3. Section 02: Frontend	18
3.1 Technologies Used	18
3.2 Design Colors	18
3.3 Typography	19

3.4 Custom Buttons	20
3.5 Custom Components	21
3.6 Route Calculation Engine	23
3.7 Coordinate Engine	23
3.8 Responsive Design	23
4. Section 03: Backend	24
4.1 Technologies Used	24
4.2 Database Schema.....	25
4.3 Database Records	25
4.3.1 Metro Lines	25
4.3.2 Fare Bands	25
4.3.3 Sample Stations	26
4.4 API Endpoints.....	26
4.5 Authentication	27
4.6 Dynamic Content	27
5. Complete Technology Summary.....	28
6. Conclusion	28
7. References.....	29

1. Introduction

MetroM is a comprehensive web application designed to simplify the Cairo Metro experience for daily commuters and occasional travelers. Cairo, one of the world's most densely populated cities, relies heavily on its metro system as the primary mode of public transportation, carrying over 3.5 million passengers daily across three operational lines. Despite its importance, navigating the metro system remains a significant challenge due to complex line interchanges, unclear fare structures, and the absence of a unified digital solution tailored specifically to the Cairo Metro network.

The MetroM project addresses these challenges by providing a user-friendly platform that enables passengers to plan their routes efficiently, calculate fares accurately, visualize station locations on an interactive map, and find the nearest metro station using GPS coordinates. Built with React for the frontend and a robust Express.js backend with Prisma ORM and SQLite for data persistence, MetroM delivers a seamless experience across all devices. The application also includes a Progressive Web App (PWA) configuration with service worker support for offline access to cached map tiles and static assets.

This report documents the complete development lifecycle of MetroM, from UX research and problem definition through frontend implementation and backend architecture. The project follows industry-standard HCI methodologies and modern software development practices, with all source code publicly available on GitHub for review and collaboration.

1.1 Project Objectives

- **Simplify Route Planning:** Enable users to find the optimal route between any two stations with clear step-by-step directions, including transfer guidance at interchange stations.
- **Provide Fare Transparency:** Display accurate fare calculations based on station count before the journey begins, including regular, elderly, and special needs pricing tiers.
- **Interactive Map Visualization:** Offer a Leaflet-based interactive map showing all metro lines, stations, and route highlighting for planned trips.
- **GPS-Based Discovery:** Help users find the nearest metro station using their current GPS coordinates, with coordinate import support for external map links.
- **Travel Mode:** Provide live GPS tracking during metro trips with real-time speed calculation and automatic nearest-station detection.

- **Trip Management:** Allow users to save, view, and export planned trips in JSON, CSV, and GeoJSON formats.

1.2 Target Platform

MetroM is built as a responsive web application using React 18, optimized for mobile viewing to serve the majority of users who access transportation apps on their smartphones. The application also functions seamlessly on desktop browsers, ensuring broad accessibility. The Vite build tool produces optimized production bundles with a base path configured for GitHub Pages deployment. A service worker caches static assets and CARTO map tiles for offline access, effectively enabling PWA capabilities without requiring installation from an app store.

2. Section 01: UX Phase

The UX phase follows a structured four-stage process: Research, Problem Definition, Solution Ideation, and Wireframing. Each stage builds upon the previous one to ensure a user-centered design approach. The insights gathered during this phase directly informed the architecture decisions, feature selection, and visual design of the final application.

2.1 Stage I: Research

The research stage aimed to understand how users navigate the Cairo Metro, identify their pain points, and determine their needs from a metro planning application. A comprehensive survey was conducted in Arabic with Cairo Metro passengers across different demographics to gather quantitative and qualitative data about their commuting experience. The survey covered topics including usage frequency, route confusion, fare awareness, search preferences, and desired app features.

2.1.1 Survey Analysis

Question	Response Distribution	Key Insight
Usage frequency	68% Daily, 20% Weekly, 8% Monthly, 4% Rarely	Majority are daily commuters who need reliable route information
Biggest problem	45% Route confusion, 30% Fare uncertainty, 15% Crowding info, 10% Other	Route planning is the primary pain point
Know route beforehand	35% Yes, 40% Sometimes, 25% No	65% of users need route assistance at least occasionally

Search preference	72% Station name, 28% Map view	Text-based station search is preferred
Fare information needed	85% Yes, 15% No	Fare transparency is critical
Use apps while traveling	90% Yes, 10% No	Mobile-first design is essential
Desired app features	55% Simple UI, 25% Offline mode, 20% Real-time updates	Simplicity is the top priority
Live guidance wanted	78% Yes, 22% No	Live trip guidance is highly desired

Table 1: Survey Results Summary

Conclusions and Gains:

- Fast route planning is essential: 68% of respondents use the metro daily and need quick access to route information, confirming that the core feature must be the route planner.
- Simple station selection: 72% prefer searching by station name over map view, indicating the need for an intuitive text-based search interface with dropdown selectors.
- Fare information is critical: 85% want to know the fare before traveling, highlighting the importance of displaying fare calculations prominently before the user begins their journey.
- Mobile-first design: 90% use mobile apps while traveling, confirming that the application must be optimized for mobile devices with responsive layouts and touch-friendly controls.
- Clear directions with minimal steps: Users expressed frustration with complex interfaces, emphasizing the need for simplicity and step-by-step guidance with visual line color indicators.

Pain Points Identified:

- Confusing routes with multiple line changes and unclear interchange stations.
- Uncertainty about correct fares and pricing tiers before boarding.
- Difficulty finding station names and their locations on the metro network.
- Lack of visual guidance for transfers between metro lines.
- No access to fare information before arriving at the ticket gate.
- Difficulty navigating in crowded conditions with limited time and attention.

2.1.2 Empathy Map

Dimension	Details
THINK	"I need to find the fastest route." "I don't want to waste time searching." "Which line should I take to reach my destination?"
FEEL	Frustrated when routes are confusing; Anxious when changing lines; Relieved when the app is simple and clear; Confident when fare information is transparent
SAY	"Which station should I take?" "How much will the trip cost?" "Is this the right line?" "Where do I transfer?"
DO	Search for station names; Compare different routes; Check fare details; Use the app while traveling; Ask fellow passengers for directions
PAIN POINTS	Too many steps to find a route; Confusing station names; Lack of live guidance; Hard to understand transfers; No fare information before travel
NEEDS	Simple route planner; Clear line colors for visual recognition; Easy station selection; Fare and direction information; Live trip guidance

Table 2: Empathy Map - Cairo Metro Passenger

2.1.3 User Persona

Attribute	
Name	Ahmed
Age	22
Occupation	University Student + Part-time Worker
Location	Cairo, Egypt
Frequency	Daily metro user
Device	Mobile-first
Goal	Travel quickly from one station to another without confusion - find the correct line, station, and fare in seconds.
Background	Uses Cairo Metro daily for university and part-time work. Needs a fast, reliable way to check the correct line, station number, and fare without switching between multiple apps or asking strangers.
Frustrations	Does not always know which line to take. Wastes time switching between apps or asking others for directions. Wants an app optimized for mobile that works in crowded conditions.
Needs	Fast and clear route search, metro line colors for quick visual recognition, simple step-by-step instructions, live map or route preview during travel.

Table 3: User Persona - Ahmed

2.2 Stage II: Define Problem

2.2.1 Pain Points Categorization

Category	Pain Points	Severity
Route Planning	Difficulty finding optimal routes, confusing transfers, multiple line changes	High (45%)
Fare Information	Uncertainty about ticket prices, no advance fare knowledge	High (30%)
Station Navigation	Hard to find station names, unclear line assignments, difficulty locating stations	Medium (15%)
User Experience	Complex interfaces, lack of mobile optimization, too many steps to get information	Medium (10%)

Table 4: Categorized Pain Points

2.2.2 5Ws and H Analysis

Question	Analysis
WHO is experiencing the problem?	Cairo Metro passengers including students, workers, tourists, and daily commuters who rely on the metro for transportation.
WHAT is the problem?	Passengers struggle to efficiently plan metro routes, calculate fares, and navigate the three-line system with its complex interchanges.
WHERE does it occur?	At metro stations, on trains, and while planning trips from home or work - primarily in mobile environments with limited time.
WHEN does it happen?	During rush hours (7-9 AM, 4-7 PM) when passengers are under time pressure and stations are most crowded.
WHY does it matter?	Inefficient navigation wastes time, causes missed connections, creates passenger anxiety, and reduces overall satisfaction with public transport.
HOW can we solve it?	Create a mobile-first web application with intuitive route planning, fare calculation, interactive map, and GPS-based station discovery.

Table 5: 5Ws and H Analysis

2.2.3 Problem Statements

Using the template: **[User] needs a way to [do something] because [insight].**

Problem Statement 1: Ahmed, a daily metro commuter, needs a way to quickly find the optimal route between any two stations because he currently wastes significant time asking others or switching between apps to figure out the correct lines and transfers.

Problem Statement 2: Ahmed needs access to accurate fare information before starting his journey because unexpected fare costs disrupt his budget planning and cause stress at ticket gates.

Problem Statement 3: Ahmed needs a simple, mobile-optimized interface that works in crowded conditions because complex apps with too many steps are impractical when he has limited time and attention while commuting.

2.3 Stage III: Ideate Solution

2.3.1 Goal Statements

Using the template: **Our [product] will let [user] [do something] so that [outcome].**

Goal 1: Our MetroM app will let Ahmed plan routes between any two stations in under 30 seconds so that he can confidently navigate the metro system without asking others or using multiple apps.

Goal 2: Our MetroM app will let Ahmed view accurate fare calculations before traveling so that he can budget his transportation expenses and avoid surprises at ticket gates.

Goal 3: Our MetroM app will let Ahmed access all metro information through a clean, mobile-optimized interface so that he can use the app effectively even in crowded stations with limited time.

Goal 4: Our MetroM app will let Ahmed find the nearest metro station using his GPS coordinates so that he can start his journey from any location without needing to know the area.

2.3.2 Proposed Solutions Comparison

Solution	Description	Pros	Cons
A. Route Planner Web App	A responsive web application focused on route planning, fare calculation, interactive map, and GPS station discovery.	Cross-platform, easy to update, no installation required, accessible to everyone with a browser.	Requires internet connection for API calls; limited offline functionality (mitigated by service worker).
B. Native Mobile App	A dedicated Android/iOS app with GPS integration, offline maps, and push notifications.	Better performance, offline access, native device features.	Higher development cost, separate codebases, app store approval required, not cross-platform.
C. Information Website	A static website with metro maps, fare tables, and general information.	Simple to build, fast loading, works offline with caching.	No personalized route planning, no dynamic content, limited user engagement.

Table 6: Proposed Solutions Analysis

Selected Solution: Solution A: Route Planner Web App was selected because it provides the best balance of functionality, accessibility, and development feasibility. As a responsive React SPA with Express.js backend, it serves both mobile and desktop users from a single codebase, requires no app store distribution, and can be easily updated. The modern web stack (React 18 + Express + Prisma + SQLite) enables dynamic content, interactive map visualization with Leaflet.js, and a smooth user experience. The addition of a service worker provides offline caching capabilities, bridging the gap between web and native app functionality.

2.3.3 Comparison with Existing Solutions

Existing Solution	Weak Points	How MetroM Outperforms
Google Maps	Limited Cairo Metro detail, inaccurate interchange information, no fare data, generic transit info.	Dedicated Cairo Metro data, accurate fare calculation (regular/elderly/special needs), line-specific routing, station-specific information.
Cairo Metro Official Website	Outdated interface, not mobile-friendly, static PDF maps, no route planning feature.	Modern responsive design, interactive Leaflet map, route planner with BFS pathfinding, mobile-first approach.
Third-Party Transit Apps	Often inaccurate for Cairo, limited station coverage, no fare information, English only.	Complete 74-station database across 3 lines, accurate fares, GPS travel mode, trip export capabilities.
Paper Maps	Static information, easily damaged, no real-time updates, difficult to carry.	Dynamic route calculation, always up-to-date via admin panel, accessible on any device, eco-friendly.

Table 7: Comparison with Existing Solutions

2.3.4 Selected Solution and App Features

Core Features:

- **Route Planner:** Select start and destination lines and stations to calculate the optimal route using BFS pathfinding with step-by-step directions, transfer guidance, and estimated travel time.
- **Fare Calculator:** Automatic fare calculation based on the number of stations traveled, displaying regular, elderly, and special needs pricing tiers.
- **Interactive Map:** Leaflet.js-based map showing all 74 stations across 3 metro lines with color-coded polylines, station markers, and route highlighting.

- **Travel Mode:** Live GPS tracking with speed calculation (GPS + Haversine-based smoothing), nearest station detection, remaining distance, and auto-stop at destination.
- **Coordinate Tools:** Import coordinates from Google Maps, Apple Maps, Waze, or OSM links; live geolocation; nearest station finder using Haversine distance formula.
- **Station Information:** Comprehensive accordion-style display of all stations organized by metro line with color-coded station pills.

User Account Features:

- **Authentication:** Secure JWT-based login and registration system with password hashing (bcryptjs) and refresh token rotation.
- **Trip Management:** Save planned trips, view trip history, and export trips in JSON, CSV, and GeoJSON formats.
- **Profile Management:** Personal account settings with name and email editing, password changes, and account deletion.

Administrative Features:

- **Admin Panel:** Full CRUD operations for metro lines, stations, and fare bands, enabling real-time data management without code changes.
- **Role-Based Access:** First registered user automatically receives ADMIN role; subsequent users get USER role with protected endpoints.

Additional Features:

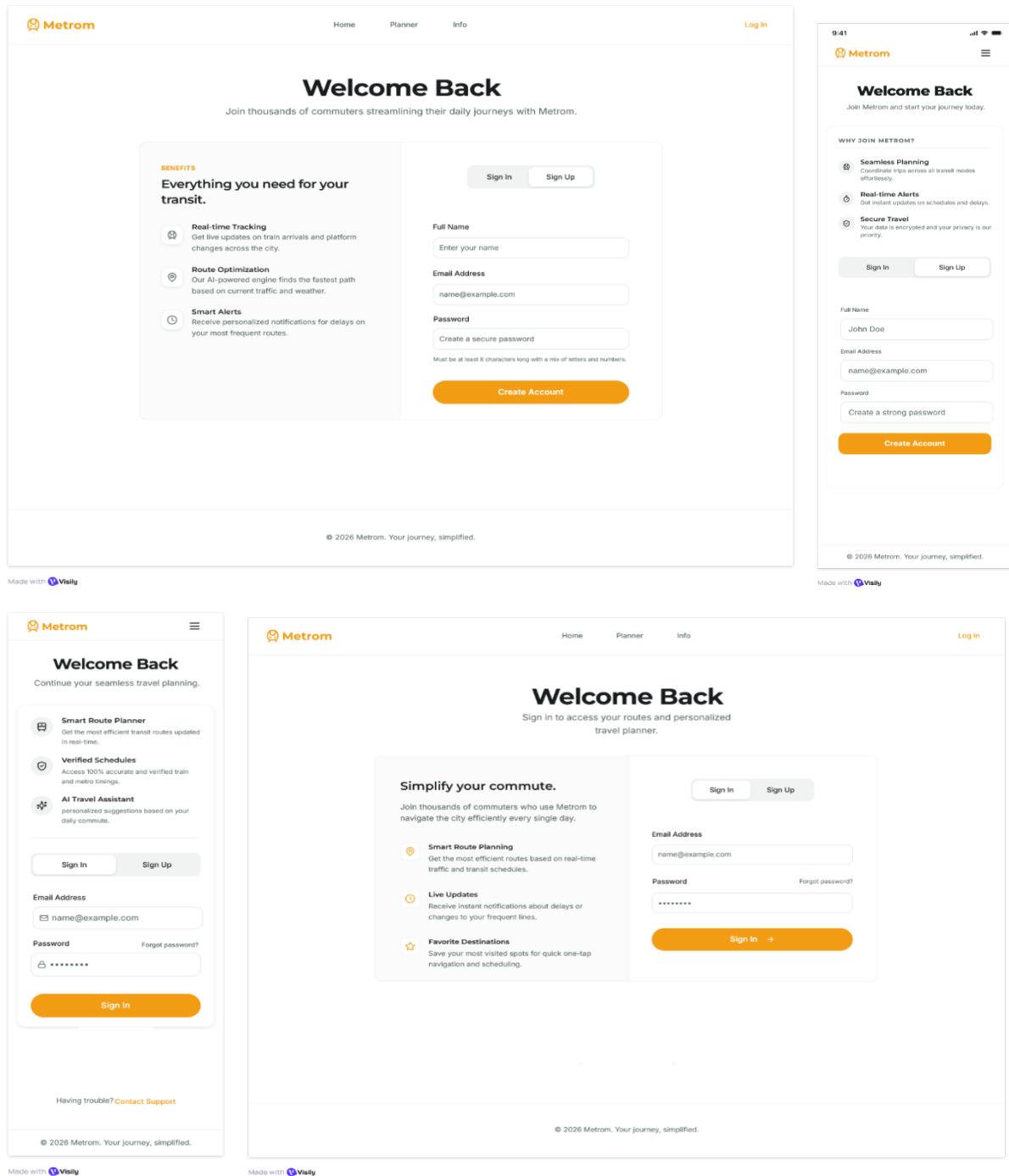
- **PWA Support:** Web manifest with full icon set (Apple, Android, MS tiles) and service worker for offline caching of static assets and map tiles.
- **Responsive Design:** Fully adaptive layout with breakpoints at 576px, 768px, and 1024px for mobile, tablet, and desktop views.
- **Parallax Hero:** Animated hero section with 9 layered illustration PNGs and pointer-based parallax movement for an engaging landing experience.

2.4 Stage IV: Wireframes and Site Map

2.4.1 Wireframes

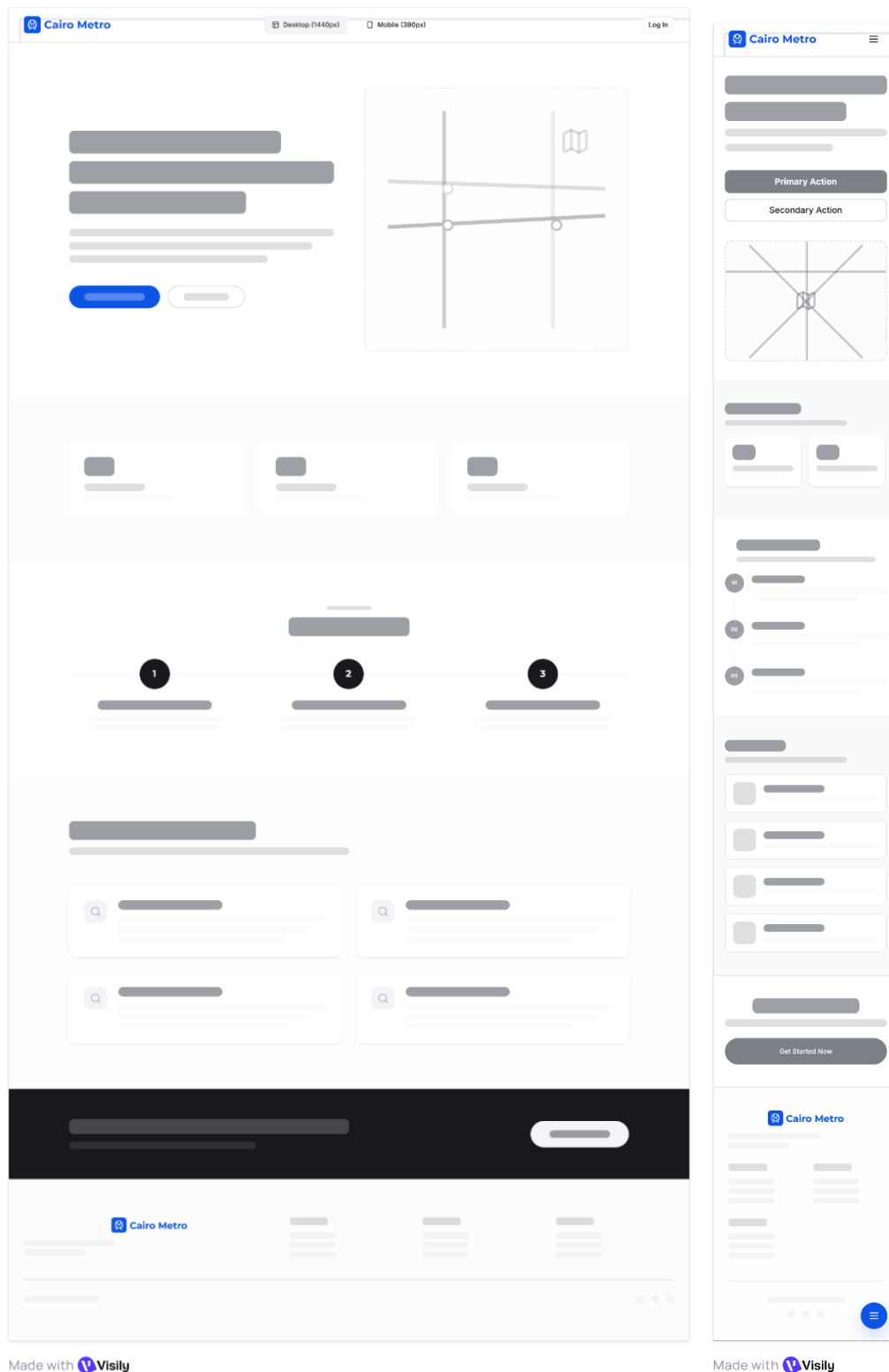
Low-fidelity wireframes were created for all key screens in the application, following a mobile-first approach. Each wireframe defines the layout elements and their purpose to ensure a consistent user experience across all screens.

1. Sign-up / Log-in Page



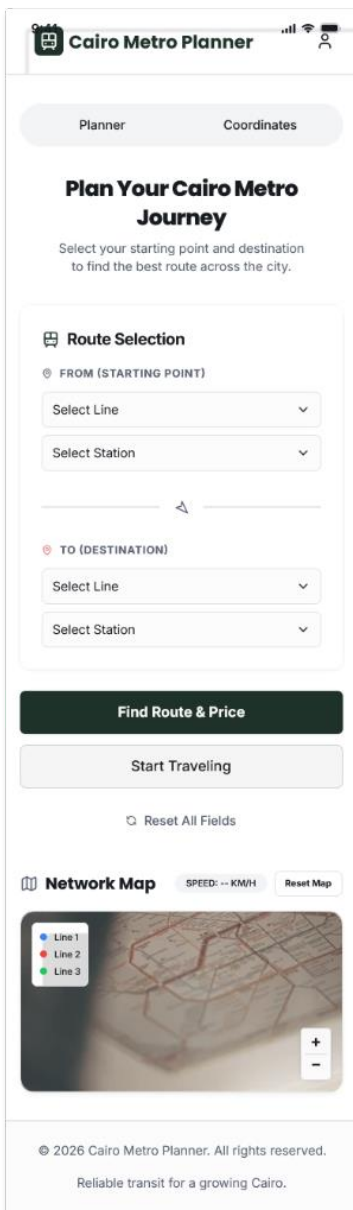
This page is the entry point for new and returning users. It provides separate sign-in and sign-up states, with fields for authentication and a short section that highlights the app's benefits. The layout is designed to build trust, guide the user quickly into the app, and keep the action area simple and focused.

3.Home Page

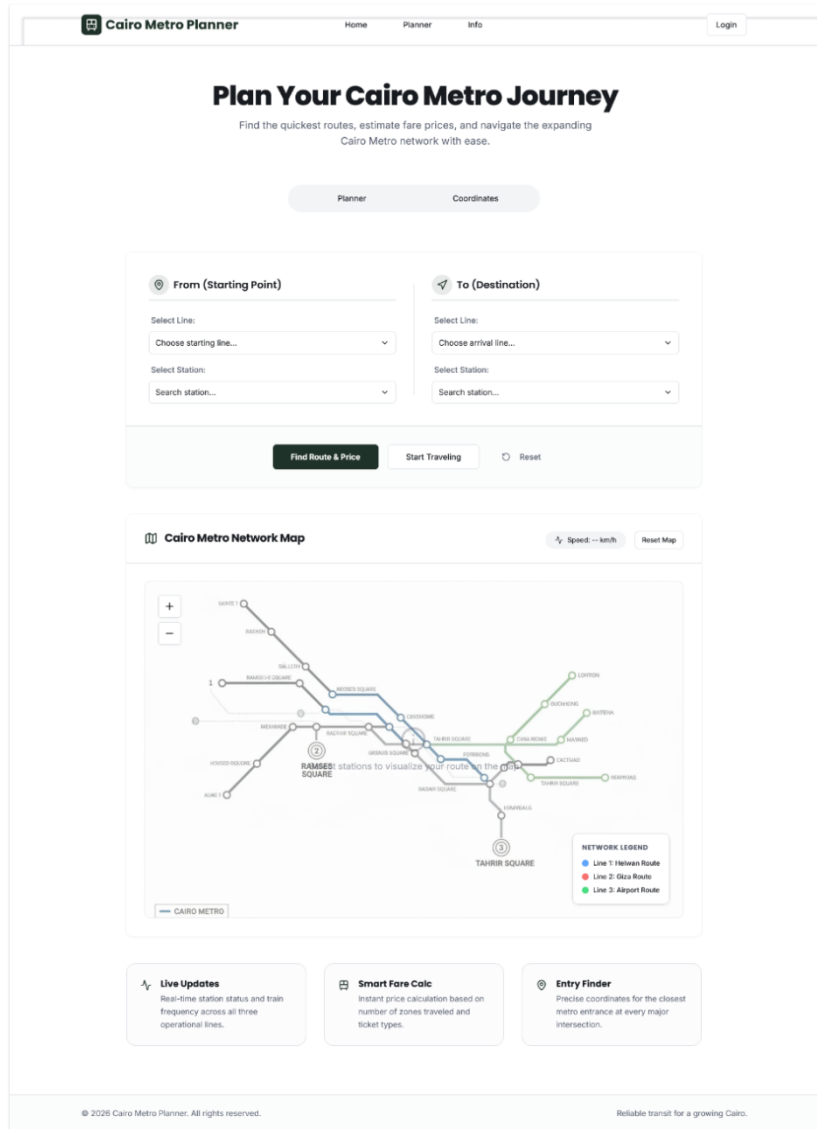


The home page is the landing screen after authentication or when a visitor opens the app. It introduces Metrom, presents a short overview of the service, and links users to the main features such as planning, metro information, and station guidance. It acts as the central navigation hub and gives users a quick understanding of what the app offers.

3. Planner Page



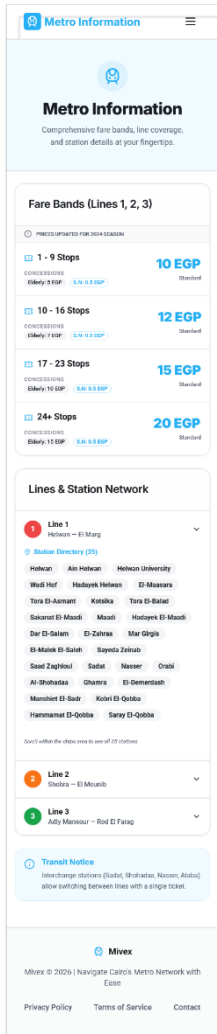
Made with Visily



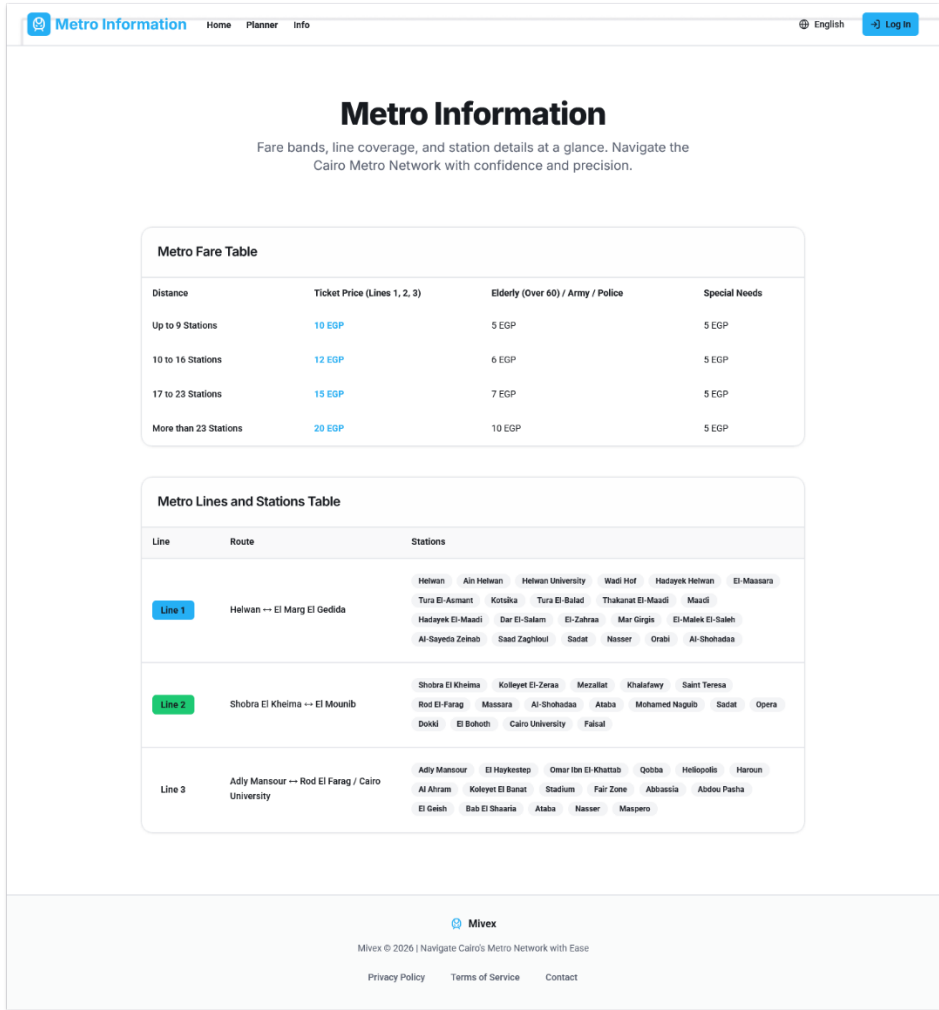
Made with Visily

The planner page is the main trip-planning workspace. It allows users to choose their starting line and station, destination line and station, and then calculate the best route and fare. This page also includes route visualization, travel actions, and supporting metro map content, making it the core functional screen of the app.

4. Information Page



Made with Visaly



Made with Visaly

The information page displays static metro reference data in a structured format. It includes fare bands, line coverage, and station lists so users can quickly check prices and network details without planning a trip. This screen supports users who only need metro information and not route calculation.

5. Coordinates page

Cairo Metro Planner

HomePlannerInfo

Login

Plan Your Cairo Metro Journey

Pinpoint the nearest stations using geographic coordinates or map links for ultimate precision.

PlannerCoordinates

Find Closest Station by Coordinates

Enter your current coordinates and destination coordinates, or use live location for instant results.

CURRENT LOCATION

LOCATION LINK

Paste Google Maps / Waze / Apple Maps link

LATITUDE

e.g. 30.0444

LONGITUDE

e.g. 31.2357

DESTINATION LOCATION

LOCATION LINK

Paste destination map link or lat, long

LATITUDE

e.g. 30.0444

LONGITUDE

e.g. 31.2357

Import Coordinates From Links

Use Live Location

Closest Station To Me

Find Closest Start & End

Closest-station results will appear here after search.
We'll show you the nearest line, station name, and distance.

Cairo Metro Network Map

Visual network overview and real-time positioning.

STATUS: Network Active

SPEED: -- km/h

Reset Map

© OpenStreetMap contributors © CARTO

Live Network Connectivity Verified

© 2026 Cairo Metro Planner. All rights reserved.

Reliable transit for a growing Cairo.

Made with Visiily

Cairo Metro Planner

Metro Coordinates

Find the closest Cairo Metro stations using precise geographic location data.

PlannerCoordinates

Closest Station Lookup

Enter map links or manual coordinates below.

ORIGIN (CURRENT)

LOCATION LINK

Paste Google/Waze link...

LATITUDE

e.g. 30.0444

LONGITUDE

e.g. 31.2357

DESTINATION

LOCATION LINK

Paste destination link...

LATITUDE

e.g. 30.0100

LONGITUDE

e.g. 31.2000

Import from Links

Use Live Location

Closest Station To Me

Find Closest Start & End

CALCULATION SUCCESSFUL

Sadat Station

Line 1 & 2 Interchange

0.8 km

Dokki Station

Line 2 (Red Line)

1.4 km

CALCULATED IN 42MS BASED ON GPS

Transit Context Map

SPEED: -- KM/H

LIVE GPS

© 2026 Cairo Metro Planner. All rights reserved.

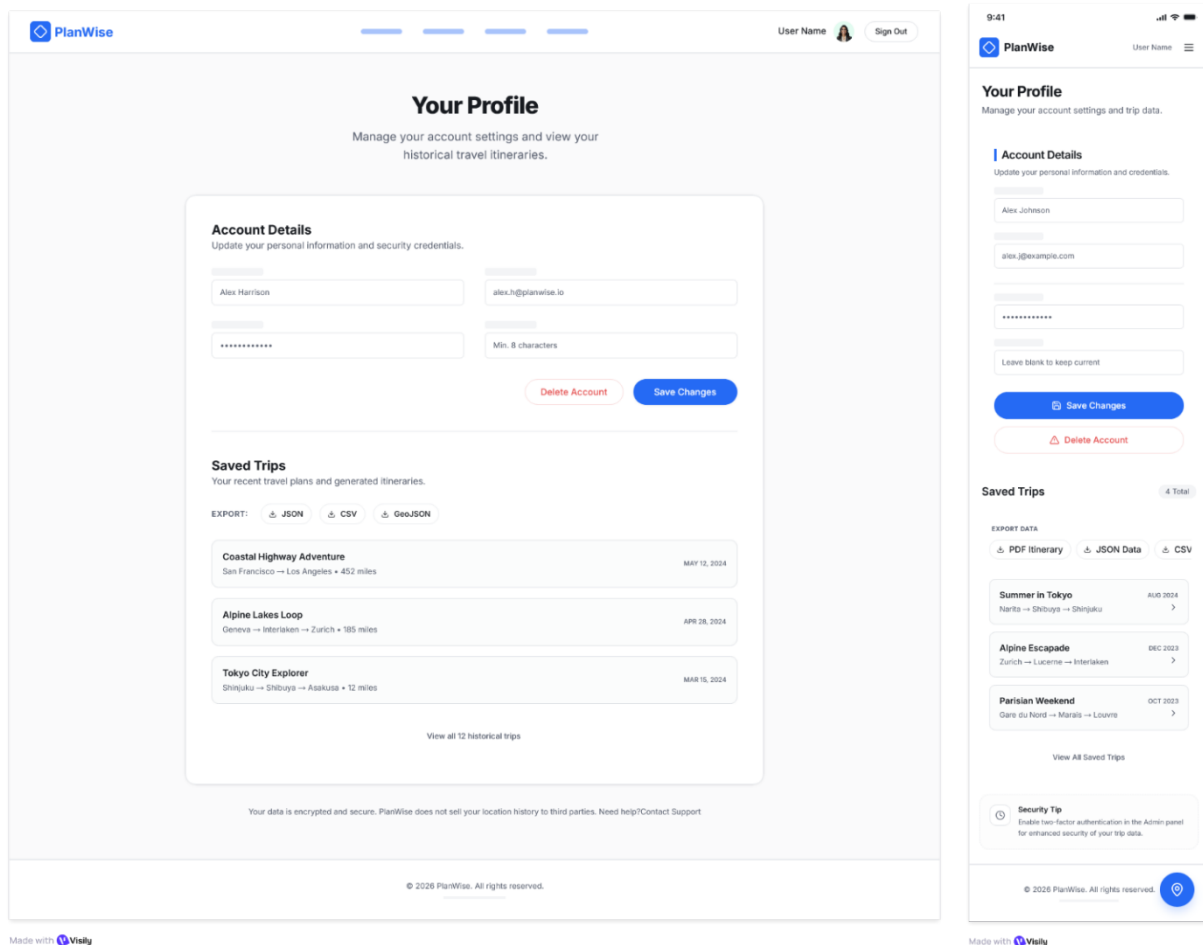
Reliable transit for a growing Cairo.

Made with Visiily

The coordinates page helps users find nearby metro stations using location data or map links. It is meant for cases where the user does not know the station name but knows their current location or destination coordinates. The page combines coordinate input, map-related tools, and nearest-station results to support location-based trip planning.

16

6.Profile



The profile page is the user account management area. It lets users update personal details, manage saved trips, and access account-related actions such as sign out or delete account. It is designed as a personal control center where users can review their data and maintain their journey history.

Link: [\[MetroM\]](#)

2.4.2 Site Map and Navigation

Screen	Type	Access From	Description
Login/Register	Auth	Entry point	Authentication gateway with login/signup forms
Home Dashboard	Tab	Root (header nav)	Overview, quick actions, recent activity
Planner	Tab	Root (header nav)	Route planning with start/end station selection
Coordinates	Sub-tab	Planner tabs	Location-based nearest station finder
Metro Info	Tab	Root (header nav)	Fare bands, station lists, metro reference
Profile	Tab	Root (header nav)	Account settings, trip history, export

Admin Panel	Link	Profile (admin only)	CRUD for metro config data
-------------	------	----------------------	----------------------------

Table 8: Navigation Structure

3. Section 02: Frontend

The frontend of MetroM is built using React 18 as a single-page application with client-side state-based navigation. All styling is implemented through custom CSS without any external CSS framework or component library. The application uses the native Fetch API for HTTP requests to the backend REST endpoints. The build process is handled by Vite 5 with a production base path configured for GitHub Pages deployment.

3.1 Technologies Used

Category	Technology	Version	Purpose
Framework	React	18.2.0	UI component library for building the SPA
Language	JavaScript (JSX)	ES2022+	Frontend application logic and component rendering
Build Tool	Vite	5.4.10	Fast development server and optimized production bundler
Styling	Custom CSS	-	Hand-written stylesheet (~2,500 lines across 3 CSS files)
Map Library	Leaflet.js	-	Interactive map rendering (bundled locally)
GeoJSON Data	Station coordinates	-	Cairo metro station points loaded locally
HTTP Client	Native Fetch API	-	REST API calls to Express backend
PWA	Service Worker + Web Manifest	-	Offline caching and installability

Table 9: Frontend Technology Stack

3.2 Design Colors

The MetroM color palette features a dark teal/green gradient background with warm gold/amber accent colors, creating a distinctive visual identity that references Cairo's metro branding while maintaining excellent readability for transit information display. The background uses a radial gradient with subtle amber highlights at the top, transitioning through deep teal tones.

Color Role	Hex Code(s)	Usage
------------	-------------	-------

Primary Accent (Gold/Amber)	#ffcf86 / #f6b94d	Headings, active navigation, primary buttons (gradient), station badges, overview values
Secondary (Dark Teal)	#0f3d35 / #123f37 / #0e332d	Body background gradient, dark theme foundation
Header	rgba(11, 43, 38, 0.78)	Sticky header with backdrop blur effect
Primary Text	#e9f5f1 / #d8ede8	Body text and navigation links on dark backgrounds
Secondary Text	#d0e4de / #b7d1cb	Supporting text, captions, secondary information
Success	#cff8dc	Success states, confirmation messages
Error	#ffd0d0	Error states, validation messages
Neutral	#9fbab4	Disabled states, placeholders, subtle borders

Table 10: Color Palette (Primary, Secondary, Neutral)

 #1E6B42 Dark Green (Primary)	 #2D9E60 Mid Green (Active)	 #4CAF7D Light Green (Success)
 #F5C842 Yellow/Gold (Fares)	 #FFFFFF White (Background)	 #8A8A85 Gray (Secondary Text)

Metro Line Colors (Map View):

Metro Line	Color Name	Hex Code
Line 1 (Helwan - New El-Marg)	Blue	#2E86DE
Line 2 (El Monib - Shubra Al Khaimah)	Red	#E74C3C
Line 3 (Adly Mansour - Ring Road)	Green	#27AE60

Table 11: Metro Line Colors

 #E43C2F Line 1 (Red)	 #1B7A43 Line 2 (Green)	 #F5A623 Line 3 (Yellow)
--	--	---

3.3 Typography

The application uses the Inter font family as its primary typeface, specified with system font fallbacks including system-ui, -apple-system, BlinkMacSystemFont, and Segoe UI. Inter provides excellent readability on mobile screens with its optimized letter spacing and x-height. No

external font files are loaded via @import or link tags; the application relies on system font availability, which ensures zero additional network requests and instant rendering.

Element	Font Family	Weight	Size	Usage
Body Text	Inter / System Fallbacks	Regular (400)	16px (1rem)	Primary content text throughout the application
Headings	Inter / System Fallbacks	Bold (700)	20-32px	Section headings, page titles, stats values
Buttons	Inter / System Fallbacks	SemiBold (600)	14-16px	Call-to-action buttons and interactive elements
Captions / Small	Inter / System Fallbacks	Regular (400)	12-14px	Labels, timestamps, secondary information

Table 12: Typography System

3.4 Custom Buttons

All buttons in the MetroM application follow consistent design principles with hand-written CSS styles. Buttons use rounded corners for a modern feel, sufficient height for comfortable mobile tapping, and clear action hierarchy using the established gold/amber accent color system on the dark teal background.

Component Name	Style	Where Used	Design Description
Primary Button	Gold gradient background (#ffcf86 to #ff6b94d), dark text, 12px border-radius, 48px height, subtle shadow	Find Route, Start Traveling, Sign In, Sign Up, Save actions	Prominent call-to-action button with gradient fill. Uses the primary gold accent color to draw attention. Includes hover scale transform and active press effect for tactile feedback.
Tab Button	Transparent background, gold text when active, muted text when inactive, bottom border indicator (2px gold)	Planner/Coordinates tabs, Bottom navigation (Home, Planner, Info, Profile)	Segmented tab navigation with underline indicator on the active tab. Active state uses gold text color (#ffcf86) with a 2px bottom border. Inactive state uses muted secondary text color.
Icon/Link Button	Transparent background, no border, gold text, underline on hover	Navigation links in header, inline text actions	Minimal button style used for secondary actions and navigation. Appears as styled text links with gold color and underline hover effect.
Form Submit Button	Gold gradient background, centered, full-width on mobile, max-width 320px on desktop	Sign In, Sign Up, Save Profile forms	Form submission button matching the primary button style but auto-sized to the form container width. Centered within the form card.

Export Button	Transparent background, teal border (#2D9E60), teal text, 10px border-radius	Export Trip (JSON, CSV, GeoJSON) in Profile section	Secondary action button with outlined style. Teal border and text provide visual distinction from primary gold buttons while maintaining the color palette.
Danger Button	Transparent background, red-tinted border, red text, small size	Delete Account action in Profile section	Destructive action button with subtle red styling. Low visual prominence to prevent accidental clicks, with confirmation step before executing the action.

Table 13: Custom Button Components

3.5 Custom Components

All UI components are hand-built React functional components organized into six categories. Each component is designed with a consistent visual language that matches the dark teal and gold color palette. The table below details each component, its styling approach, where it is used in the application, and its design rationale.

Component	Category	Style / Design	Where Used
App.jsx	Root	State manager using useState for activePage and auth. No visible UI; wraps all child components with conditional rendering based on authentication state.	Application root - wraps all pages
SiteHeader.jsx	Layout	Sticky positioned header with rgba(11,43,38,0.78) background, backdrop blur (14px), gold text (#ffc86). Collapses to hamburger menu on mobile (< 768px). Navigation links with gold hover effect.	All pages - persistent top navigation
SiteFooter.jsx	Layout	Simple centered footer with muted text, copyright notice, and optional links. Minimal height with top border separator.	All pages - bottom of every view
HeroSection.jsx	Section	Full-width parallax hero with 9 layered PNG illustrations. Pointer tracking moves layers for depth effect. Gold heading text, dark CTA button, animated entrance.	Home page - landing screen
HomeOverviewSection.jsx	Section	Three stat cards (3 lines, 74 stations, 4 fare bands) in a responsive grid. Each card has teal background, gold value text, and muted label. Below: 3-step "How it works" with numbered circles.	Home page - below hero

FeatureSection.jsx	Section	Grid of 4 feature cards (Route Planner, Fare Info, Interactive Map, GPS Finder). Each card has icon, title, description, teal background with hover lift effect.	Home page - overview section
RoutePlanner.jsx	Planner	Container component managing PlannerTabsNav, PlannerTripForm, CoordinateTools, and MetroMapSection. Switches between planner and coordinate sub-views.	Planner page - main orchestrator
PlannerTabsNav.jsx	Planner	Two tab buttons ("Planner" / "Coordinates") with active gold indicator. Full-width bar with centered tabs, 2px bottom border on active tab.	Planner page - tab navigation
PlannerTripForm.jsx	Planner	Form with 4 custom dropdowns (start line, start station, end line, end station) styled with teal borders and gold focus states. Results display: route badges (station pills), fare card (gold accent), duration, stops count, transfer guide.	Planner page - route form and results
CoordinateTools.jsx	Planner	Text input for pasting map links (Google/Apple/Waze/OSM), "Use My Location" button, nearest station list with Haversine distance. Form elements use consistent teal styling.	Planner page - coordinates sub-tab
MetroInfo.jsx	Planner	Collapsible fare bands table (4 tiers with regular/elderly/special needs columns). Accordion of 3 metro lines, each expanding to show color-coded station pills.	Info page - fare table and lines
MetroMapSection.jsx	Map	Leaflet.js full-width map with CARTO tiles. Station markers colored by line (blue/red/green). Polygons connecting stations. Route highlighting (purple). Start (green) and end (red) markers. Travel mode with GPS tracking.	Planner page - map view
AuthSection.jsx	Auth	Shared component for Sign In and Sign Up with mode toggle. Form card with email/password inputs (teal borders), gold gradient submit button, centered layout. Error messages in red-tinted style.	Auth page - login and registration
ProfileSection.jsx	Auth	Two-column layout: edit form (name, email, password) and saved trips list. Export buttons (JSON, CSV, GeoJSON) in outlined teal style. Delete account danger button with red styling.	Profile page - account management

AdminPanel.js x	Admin	Tabbed interface (Lines / Stations / Fare Bands). Each tab has a list view and add/edit form. Tables with teal headers, alternating row colors. CRUD buttons with gold primary and teal secondary styling.	Admin page (admin role only)
--------------------	-------	--	------------------------------

Table 14: Custom React Components

3.6 Route Calculation Engine

The route calculation logic is implemented in **plannerEngine.js**, a client-side JavaScript module that uses Breadth-First Search (BFS) pathfinding to compute optimal routes between any two stations. The engine maintains an interchange matrix defining transfer points between metro lines: Line 1 and Line 2 connect at Al Shohadaa and Sadat stations; Line 1 and Line 3 connect at Nasser station; Line 2 and Line 3 connect at Ataba station. For cross-line routes, the engine identifies the shortest path through interchange stations, counts total stops, calculates estimated travel time, and looks up the appropriate fare band from the database. Line 3 has special handling for branching segments (main line from Adly Mansour to Kit Kat, plus extensions to Rod El Farag Corridor and Cairo University).

3.7 Coordinate Engine

The coordinate functionality is implemented in **coordinateEngine.js**, which provides tools for parsing coordinate links from Google Maps, Apple Maps, Waze, and OpenStreet Map URLs, extracting latitude and longitude values from various URL formats. The module also implements the Haversine formula for calculating the great-circle distance between two geographic coordinates, which is used to determine the nearest metro station to a given point. This enables users to find their closest metro station either by importing a shared location link or by using the browser's Geolocation API for live positioning.

3.8 Responsive Design

MetroM implements a fully responsive design that adapts seamlessly across all device sizes using CSS media queries at three breakpoints. The mobile-first approach ensures optimal experience on smartphones, while the layout gracefully expands for tablets and desktops.

Breakpoint	Target Devices	Layout Adaptations
Base (< 576px)	Small smartphones	Single column layout, full-width buttons, stacked form fields, dropdown mobile menu
576px+	Large smartphones	Slightly increased padding, wider touch targets, improved spacing

768px+	Tablets	Multi-column grids for info pages, expanded content width, improved form layout
1024px+	Desktops	Maximum content width centered, prominent navigation, expanded content areas

Table 15: Responsive Breakpoints

Responsive Design Examples: See the placeholder screenshots in Section 3.8 above for visual demonstrations of the application on mobile (375px) and desktop (1024px+) viewport widths.

4. Section 03: Backend

The MetroM backend provides a RESTful API layer built with Express.js and TypeScript, connecting the frontend to a SQLite database managed through Prisma ORM. The backend handles user authentication with JWT tokens, manages saved trips, and provides both public metro configuration data and admin-only CRUD endpoints for managing metro lines, stations, and fare bands. The server runs on port 4000 and is accessed by the frontend through a Vite development proxy configured at the /api path.

4.1 Technologies Used

Category	Technology	Version	Purpose
Framework	Express	4.21.1	Fast, lightweight web framework for REST API
Language	TypeScript	5.6.3	Type-safe backend development
Runtime	tsx	4.19.2	TypeScript execution and watch mode for development
ORM	Prisma	5.22.0	Type-safe database queries and schema management
Database	SQLite	-	File-based relational database (dev.db)
Authentication	JWT (jsonwebtoken)	9.0.2	Access and refresh token generation and verification
Password Hashing	bcryptjs	2.4.3	Secure password hashing with 10 salt rounds
Validation	Zod	3.23.8	Runtime type validation for request bodies
CORS	cors	2.8.5	Cross-origin resource sharing for frontend communication
Environment	dotenv	16.4.5	Environment variable management

Table 16: Backend Technology Stack

4.2 Database Schema

The database consists of six interconnected tables managed by Prisma ORM, storing all application data including user accounts, authentication tokens, saved trips, metro configuration, and fare pricing:

Model	Description	Key Fields
User	User accounts and authentication	id (cuid), name, email (unique), passwordHash, role (USER/ADMIN), timestamps
RefreshToken	JWT refresh token storage	id, userId (FK), expiresAt, revoked (bool), createdAt
Trip	Saved user trips with route data	id (cuid), userId (FK), startLine, endLine, startStation, endStation, stationsCount, estimatedTime, fares (JSON), route (JSON), status
MetroLine	Metro line definitions	id (string), label, routeLabel, timestamps; has many MetroStation
MetroStation	Individual station records	id (cuid), lineId (FK), name, sortOrder (int), timestamps; unique [lineId, name]
FareBand	Fare pricing tiers	id (cuid), distance, minStations, maxStations, ticket, elderly, specialNeeds, timestamps

Table 17: Database Schema Overview

4.3 Database Records

4.3.1 Metro Lines

Label	Route	Station Count	Key Interchanges
Line 1	Helwan - New El-Marg	35 stations	Sadat (Line 2), Al Shohadaa (Line 2), Nasser (Line 3)
Line 2	El Monib - Shubra Al Khaimah	20 stations	Sadat (Line 1), Al Shohadaa (Line 1), Ataba (Line 3)
Line 3	Adly Mansour - Ring Road	29 stations (+ extensions)	Nasser (Line 1), Ataba (Line 2), Kit Kat (branching hub)

Table 18: Metro Lines Summary (74 total stations)

4.3.2 Fare Bands

Distance	Min Stations	Max Stations	Regular (EGP)	Elderly (EGP)	Special Needs (EGP)
Band 1	1	9	8	4	5
Band 2	10	16	10	5	5

Band 3	17	23	15	8	5
Band 4	24	39	20	10	5

Table 19: Fare Bands (Egyptian Pounds)

4.3.3 Sample Stations

id	Station Name	Line	Order	Is Interchange
1	Helwan	Line 1	1	No
11	Maadi	Line 1	11	No
19	Sadat	Line 1	19	Yes (Line 2)
40	El-Mounib	Line 2	1	No
49	Cairo University	Line 2	6	No
53	Ataba	Line 2	14	Yes (Line 3)
63	Abbassiya	Line 3	4	No

Table 20: Sample Station Records from Database

4.4 API Endpoints

Route	Endpoint	Method	Auth	Description
Health	/health	GET	No	Health check returning service status
Auth	/auth/signup	POST	No	Create account (first user becomes ADMIN)
Auth	/auth/signin	POST	No	Sign in with email and password
Auth	/auth/refresh	POST	No	Refresh access token (rotates refresh tokens)
Auth	/auth/signout	POST	No	Revoke refresh token
Users	/users/me	GET	Yes	Get current user profile
Users	/users/me	PATCH	Yes	Update name, email, or password
Users	/users/me	DELETE	Yes	Delete account (cascades trips and tokens)
Trips	/trips	GET/POST	Yes	List / Save user trips
Trips	/trips/:id	GET/PATCH/DELETE	Yes	View / Update / Delete trip (owner only)
Metro	/metro/config	GET	No	Public: lines with stations and fare bands
Metro	/metro/admin/*	CRUD	Admin	Admin CRUD for lines, stations, and fare bands

Table 21: API Endpoints

4.5 Authentication

MetroM implements a secure authentication system using JSON Web Tokens (JWT) with a dual-token strategy. The system uses short-lived access tokens (15 minutes) for API authentication and long-lived refresh tokens (7 days) stored in the database for session persistence. Passwords are hashed using bcryptjs with 10 salt rounds before storage. The AuthSection.jsx component serves as a shared custom component for both sign in and sign up, toggling between the two modes without navigating to a separate page.

- User submits email and password through the shared AuthSection component.
- The backend verifies credentials against the bcryptjs hash in the SQLite database.
- On success, the server issues an access token and a refresh token.
- The frontend stores the session in localStorage (key: "metrom-session") and restores it on app load.
- The requireAuth middleware validates the Bearer token on protected endpoints.
- The requireAdmin middleware additionally verifies role === "ADMIN" for admin endpoints.
- The first user to sign up automatically receives the ADMIN role for immediate admin panel access.

4.6 Dynamic Content

All metro configuration information (lines, stations, fare bands) is stored in the SQLite database and fetched through Express REST API calls using the native Fetch API. When an administrator modifies data through the Admin Panel, changes are immediately persisted and reflected across all frontend views on the next data fetch. This architecture enables real-time updates without code changes.

Content Area	Data Source	Update Mechanism
Station Lists (Planner)	metro_stations via GET /metro/config	Fetched on page load, grouped by line
Route Calculation	Client-side plannerEngine.js	Calculated on-demand based on user selection
Fare Display	fare_bands via GET /metro/config	Dynamic lookup based on station count
Trip History	trips via GET /trips (authenticated)	User-specific, updated on save/delete
Metro Info Page	metro_lines + metro_stations + fare_bands	Fetched on navigation to Info page
Admin CRUD	Full CRUD via /metro/admin/* endpoints	Changes propagate to all views on next fetch

Table 22: Dynamic Content Sources

5. Complete Technology Summary

Layer	Technology	Version	Purpose
Frontend UI	React	18.2.0	Component-based user interface
Frontend Language	JavaScript (JSX)	ES2022+	Application logic and rendering
Frontend Build	Vite	5.4.10	Development server and production bundler
Frontend Styling	Custom CSS	-	Hand-written stylesheets (~2,500 lines)
Frontend Map	Leaflet.js	-	Interactive map with markers and polylines
Frontend PWA	Service Worker	-	Offline caching and asset management
Backend Framework	Express	4.21.1	REST API web framework
Backend Language	TypeScript	5.6.3	Type-safe server development
Backend ORM	Prisma	5.22.0	Database queries and schema management
Database	SQLite	-	File-based relational data storage
Authentication	JWT (jsonwebtoken)	9.0.2	Access and refresh tokens
Password Security	bcryptjs	2.4.3	Password hashing (10 salt rounds)
Validation	Zod	3.23.8	Request body validation
HTTP Client	Native Fetch API	-	Frontend-backend communication

Table 23: Complete Frontend and Backend Technology Stack

6. Conclusion

MetroM represents a comprehensive solution to the challenges faced by Cairo Metro passengers. Through a structured UX research process, the team identified key pain points including route confusion, fare uncertainty, and the need for a mobile-optimized interface. These insights directly informed the design and development of the application, resulting in a feature-rich platform that addresses real commuter needs.

References

1. Cairo Metro Authority. (2024). Official Metro Network Statistics. Egyptian Ministry of Transport.
2. React Team. (2024). React Documentation. Meta Platforms. Retrieved from <https://react.dev/>
3. Express.js. (2024). Express Documentation. Retrieved from <https://expressjs.com/>
4. Prisma Team. (2024). Prisma ORM Documentation. Retrieved from <https://www.prisma.io/>
5. Vite. (2024). Vite Documentation. Retrieved from <https://vitejs.dev/>
6. Leaflet. (2024). Leaflet.js Documentation. Retrieved from <https://leafletjs.com/>
7. SQLite. (2024). SQLite Documentation. Retrieved from <https://www.sqlite.org/>
8. JWT. (2024). JSON Web Tokens. Retrieved from <https://jwt.io/>
9. bcryptjs. (2024). bcryptjs Documentation. Retrieved from <https://github.com/dcodeIO/bcrypt.js/>
10. Zod. (2024). Zod Documentation. Retrieved from <https://zod.dev/>